

Tag 1

Erste Schritte in Python

Variablen, Datentypen, Rechnen & Text

Was du heute lernst

1. Was ist Programmierung überhaupt?
2. Was ist eine **Variable**?
3. Die vier **Basis-Datentypen**
4. Wie man mit Variablen **rechnet**
5. Wie man Text und Zahlen zu einem **Satz** verbindet

Block 1

Was ist Programmierung?

Bild: Der Computer als Koch

Ein Computer führt Anweisungen **stur nacheinander** aus –
genau wie ein Koch, der ein Rezept abarbeitet.

Steht im Rezept (Code) ein Fehler,
schmeckt das Essen nicht (**Programmabsturz**).

Was ist Python?

Eine Programmiersprache, die sehr **nah an der englischen Schriftsprache** ist.

Dadurch ist sie besonders **leicht zu lesen** und ein guter Einstieg in die Programmierung.

Dein erster Befehl: print()

```
print("Hallo Welt")
```

Die Ausgabe erscheint **direkt unter der Zelle**:

```
Hallo Welt
```

`print(...)` zeigt dir das, was in den Klammern steht.

Auch Zahlen und Namen funktionieren

```
print(25)  
print("Sabine")
```

`print()` kann **Zahlen** genauso ausgeben
wie **Text** (in Anführungszeichen).

Variablen

Die Umzugskartons deines Codes

Bild: Der beschriftete Umzugskarton

Eine Variable ist wie ein **beschrifteter Umzugskarton**.

Du legst einen **Wert** hinein
und schreibst einen **Namen** darauf.

```
alter = 25  
name = "Sabine"
```

Eine Variable ausgeben

```
alter = 25  
name = "Sabine"  
  
print(alter)  
print(name)
```

Du sprichst die Variable einfach mit ihrem
Namen an – ohne Anführungszeichen!

Sieben typische Anfängerfehler

bei Variablennamen

Fehler 1: Anführungszeichen um den Namen

```
"abteilung" = "IT-Support" # ✗
```

✓ Richtig:

```
abteilung = "IT-Support"
```

Der **Name links** vom `=` bekommt **keine** Anführungszeichen –
nur der **Text rechts** davon!

Fehler 2: Fehlende Anführungszeichen beim Text

```
abteilung = IT-Support # ❌
```

Python sucht jetzt nach einer Variable `IT`
und interpretiert `-` als **Minus!**

✅ Richtig:

```
abteilung = "IT-Support"
```

Fehler 3: Groß-/Kleinschreibung

```
abteilung = "IT-Support"
```

```
print(abteilung)    #  funktioniert  
print(Abteilung)    #  NameError
```

`abteilung` und `Abteilung` sind für Python
zwei komplett verschiedene Variablen!

Fehler 4: Leerzeichen im Namen

```
meine abteilung = "IT-Support"  # ✗
```

✓ Richtig – nutze einen **Unterstrich**:

```
meine_abteilung = "IT-Support"
```

Fehler 5: Bindestrich im Namen

```
it-support = "IT-Support"  # ✗
```

Der Bindestrich wird wieder als **Minus** gelesen!

✓ Richtig:

```
it_support = "IT-Support"
```

Fehler 6: Variable zu früh benutzen

```
print(abteilung)      # ✗ gibt es noch nicht!  
abteilung = "IT-Support"
```

✓ Richtig – erst anlegen, dann benutzen:

```
abteilung = "IT-Support"  
print(abteilung)
```

Fehler 7: Reservierte Wörter

```
class = "IT-Support" # ✗
```

`class` ist ein **reserviertes Wort** –

Python nutzt es selbst für eine eigene Funktion.

✓ Richtig: einen eigenen, unbenutzten Namen wählen:

```
abteilung = "IT-Support"
```

Einfache oder doppelte Anführungszeichen?

Beide sind gleichwertig!

```
abteilung = "IT-Support"
```

ist dasselbe wie

```
abteilung = 'IT-Support'
```

Python behandelt ' ' und " " völlig gleich.

Wann nimmt man die andere Sorte?

Wenn der Text selbst schon Anführungszeichen enthält!

```
text = "Sophies Abteilung heißt 'IT-Support'."
```

```
text = 'Er sagte: "Hallo!"'
```

Merksatz

**Wichtig ist nur:
Anfang und Ende müssen
mit demselben Zeichen geschrieben werden.**



Block 2

Die vier Basis-Datentypen

Text (String)

```
abteilung = "IT-Support"
```

Muss **immer** in Anführungszeichen stehen.

Ganzzahl (Integer)

```
tickets_geloest = 8
```

Für **zählbare** Dinge – ohne Nachkommastellen.

Nachkommazahl (Float)

```
preis = 49.99
```

⚠ Wichtig: Python nutzt einen **Punkt**,
kein Komma!

Wahrheitswert (Boolean)

```
server_aktiv = True
```

Nur **zwei** mögliche Zustände: `True` oder `False`

⚠ Wichtig: **groß geschrieben!**

Die vier Datentypen im Überblick

Typ	Beispiel
Text (String)	"IT-Support"
Ganzzahl (Integer)	8
Nachkommazahl (Float)	49.99
Wahrheitswert (Boolean)	True / False

Datentyp herausfinden mit type()

```
a = 10
b = "Hallo"
c = 3.14

print(type(a))    # <class 'int'>
print(type(b))    # <class 'str'>
print(type(c))    # <class 'float'>
```

`type(...)` verrät dir, **welcher Datentyp**
sich hinter einer Variable verbirgt.

Achtung: Zahl oder Text?

```
zahl1 = 5  
zahl2 = "5"  
  
print(type(zahl1))    # <class 'int'>  
print(type(zahl2))    # <class 'str'>
```

Beide **sehen** gleich aus – 5 –

aber `zahl2` steht in Anführungszeichen und
ist deshalb **Text**, keine Zahl!

Block 3

Rechnen mit Variablen

Eine Variable kann ein Ergebnis speichern

```
grundpreis = 100
neuer_preis = grundpreis + 20

print(neuer_preis)
```

grundpreis + 20 → 120 → neuer_preis
 (Rechnung) (Ergebnis) (Box)

Merke: Die rechte Seite wird **zuerst** berechnet.
Das Ergebnis wandert danach in die Variable.

Die vier Grundrechenarten

Zeichen	Bedeutung	Beispiel	Ergebnis
<code>+</code>	Addition	<code>5 + 3</code>	<code>8</code>
<code>-</code>	Subtraktion	<code>5 - 3</code>	<code>2</code>
<code>*</code>	Multiplikation	<code>5 * 3</code>	<code>15</code>
<code>/</code>	Division	<code>5 / 3</code>	<code>1.666...</code>

Python nutzt `*` statt `×` und `/` statt `÷`.

Formeln in zwei Schritten übersetzen

Schritt 1: Formel in Worten aufschreiben

Fläche = Breite × Länge

Schritt 2: Wörter durch Variablennamen ersetzen

flaeche = breite * laenge

Beispiel: Fläche berechnen

```
breite = 5  
laenge = 10  
  
flaeche = breite * laenge  
print(flaeche)
```

Genau nach dem Muster von der letzten Folie:
erst die Formel, dann der Code.

Umrechnungen: kleine in große Einheit

Faustregel: Wie viele kleine Einheiten passen in eine große? → das ist der Multiplikator.

1 Stunde = 60 Minuten → `minuten = stunden * 60`

```
stunden = 2
minuten = stunden * 60
print(minuten)    # 120
```

Prozentrechnung: Prozent als Dezimalzahl

Prozent	Als Dezimalzahl
10 %	0.10
15 %	0.15
19 %	0.19

Zwei Schritte zum Endpreis

```
preis = 200

# Schritt 1: Rabattbetrag berechnen
rabattbetrag = preis * 0.15

# Schritt 2: Rabatt vom Preis abziehen
endpreis = preis - rabattbetrag
```



Warum Zwischenschritte sinnvoll sind

Jeder Rechenschritt bekommt einen **eigenen Namen**.

So bleibt dein Code **leicht lesbar**,
und du kannst jeden Schritt einzeln
mit `print()` **kontrollieren**.

Vom Aufgabentext zum Code

Die vier Übersetzungsschritte

Der Ablauf, der immer funktioniert

① Text lesen



② Formel finden



③ Variablen benennen



④ Code schreiben

Schritt ①: Zahlen im Text markieren

*"Ein Raum hat eine Breite von 5 Metern
und eine Länge von 8 Metern.
Berechnen Sie die Fläche."*

- Breite = 5
- Länge = 8
- Gesucht: **Fläche**

Schritt ②: Die passende Formel finden

Schlüsselwort im Text	Rechenart
"Summe", "zusammen"	+
"Differenz", "Rabatt"	-
"Fläche", "Volumen"	*
"aufteilen", "pro Person"	/

Schritt ③: Variablennamen vergeben

Text	Variablennamen	Wert
Breite	<code>breite</code>	5
Länge	<code>laenge</code>	8
Fläche (Ergebnis)	<code>flaeche</code>	wird berechnet

💡 Tipp: Nimm das deutsche Wort aus der Aufgabe – so findest du dich im Code sofort zurecht.

Schritt ④: Tabelle in Code übertragen

```
breite = 5  
laenge = 8  
flaeche = breite * laenge  
print(flache)
```

Die Tabelle von eben – 1:1 in Python übersetzt.

Checkliste vor dem Ausführen

- ☐ Habe ich für **jede Zahl** aus dem Text eine Variable?
- ☐ Passt mein Operator (+ - * /) zur Aufgabe?
- ☐ Ist die **Reihenfolge** richtig? (z. B. erst Rabatt berechnen, dann abziehen)
- ☐ Steht am Ende ein `print()` ?



Kurze Pause

Block 4

Text und Variablen verbinden

Drei Wege zum Ziel

Komma `,` → einfachste Methode,
automatisches Leerzeichen

Plus `+` → "Zusammenkleben",
Leerzeichen selbst setzen

f-String `f"""` → modernste Methode,
Variablen direkt in `{ }`

Methode 1: das Komma

```
name = "Michael"  
print("Guten Tag,", name)
```

"Guten Tag," , name
 ↓
 Guten Tag, Michael

Python setzt **automatisch ein Leerzeichen**
zwischen den durch Komma getrennten Teilen.

Methode 2: der Plus-Operator

```
vorname = "Lisa"
nachname = "Schmid"

print(vorname + nachname)      # LisaSchmid ❌
print(vorname + " " + nachname) # Lisa Schmid ✅
```

⚠️ Beim `+` fügt Python **kein** Leerzeichen automatisch ein – du musst es selbst setzen!

Die große Falle: Text + Zahl

```
alter = 30  
print("Ich bin " + alter)    # ✗ TypeError!
```

+ kann nur **gleiche Datentypen** verbinden.

Text und Zahl lassen sich mit + **nicht**
einfach zusammenkleben!

Funktioniert das oder nicht?

Ausdruck	Funktioniert?
"Text" + "Text"	✓
5 + 3	✓
"Text" + 5	✗

Methode 3: der f-String (die moderne Lösung)

```
benutzer = "Felix"  
anzahl_tickets = 4  
  
print(f"Hallo {benutzer}, du hast heute {anzahl_tickets} Tickets gelöst.")
```

Ein `f` vor den Anführungszeichen –
und Variablen kommen einfach in `{geschweifte Klammern}` .

So baust du einen f-String

Schritt 1: `f` vor die Anführungszeichen setzen → `f"..."`

Schritt 2: Variablen in geschweifte Klammern packen → `{variable}`

Schritt 3: Alles andere bleibt normaler Text

Der große Vorteil von f-Strings

f-Strings verbinden Text und Zahlen

ohne Fehler –

der `TypeError` von Methode 2
tritt hier **niemals** auf!

Alle drei Methoden im Vergleich

Methode	Leerzeichen?	Text + Zahl mischen?
Komma <code>,</code>	automatisch ✓	problemlos ✓
Plus <code>+</code>	selbst setzen ⚠	nur mit Umweg ✗
f-String <code>f"""</code>	wie normaler Text	problemlos ✓

🏆 **Empfehlung:** Für das Mischen von Text und Zahlen ist der **f-String** fast immer die beste Wahl.

Jetzt du: viel Übung!

Was dich erwartet

Zu jedem Block gibt es passende "**Du bist dran**"-Aufgaben:

- Eigene Variablen anlegen und ausgeben
- Datentypen erkennen und korrigieren
- Formeln aus Textaufgaben in Code übersetzen
- Text und Zahlen mit allen drei Methoden verbinden

Alle Aufgaben findest du im **Colab-Notebook**.

Was du heute gelernt hast

- ✓ Eine **Variable** ist ein beschrifteter Umzugskarton für einen Wert
- ✓ Die vier Basis-Datentypen: **String, Integer, Float, Boolean**
- ✓ Formeln lassen sich in **vier Schritten** in Code übersetzen
- ✓ Text und Zahlen verbindest du am besten mit einem **f-String**

Morgen geht's weiter: Benutzereingaben mit `input()` 🎉